



DESIGN (E) 314 TECHNICAL REPORT

Smart Prepaid Energy Controller

Author:
Utroff Rosant

Student Number:
26948982

18/05/2025

Plagiaatverklaring / Plagiarism Declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektuele eiendom van ander persone asof dit jou eie werk is.
Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.
2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is.
I agree that plagiarism is a punishable offence because it constitutes theft.
3. Ek verstaan ook dat direkte vertalings plagiaat is.
I also understand that direct translations are plagiarism.
4. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelike aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is.
Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.
5. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie.
I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.



Handtekening / Signature

UVS Rosant

Voorletters en van / Initials and surname

26948982

Studentenommer / Student number

18/05/2025

Datum / Date

Abstract

For house owners who want to control and monitor their tenants electricity usage using an integrated device called the Smart prepaid Energy Controller or SPEC. It is designed around the MCU, STM32G0B1RE microprocessor and included security using RFID prepaid unit management, as well as load control using a relay, real-time energy tracking with RTC and SD-card. A OLED display and keypad interface for users to traverse menus.

The software side includes UART communication, ADC sampling and an FSM. The hardware consists of V/I sensing circuits, Relay circuit for load simulating LED and external peripherals for logging and authentication e.g SD-card and RFID. The following report shows the hardware design, software design, and measurements taken of the prototype built.

Contents

1	Introduction	7
2	System description	7
2.1	System overview	7
2.2	System requirements.....	7
3	Hardware design and implementation	8
3.1	Hardware Block Diagram and Description of Interaction.....	8
3.2	Power Supply	9
3.3	LEDs (Debug):.....	10
3.4	Relay and control:	10
3.5	Op-amp configuration and design	11
3.6	Emergency Load Button	12
3.7	Keypad	12
3.8	OLED	12
3.9	SD Card.....	13
3.10	RFID reader (MFRC522).....	13
4	Software design and implementation	13
4.1	Software Block diagram and description of interaction.....	13
4.2	Button bounce handling and emergency S3 button	14
4.3	UART communications (protocol and timing):.....	15
4.4	ADC, Setup, Calibration, and Processing.....	17
4.5	OLED	18
4.6	FSM	20
4.7	Real-Time Clock and Energy calculations	21
4.8	SD-card setup and Commands	22
4.9	RFID- interface.....	23
5	Measurements and Results	23
5.1	Power supply measurements.....	23
5.2	UART measurements	24
5.3	Voltage and Current circuit Measurements.....	24
5.4	Calibration test for ADC	25
5.5	Voltage, current, Phase, P, Q and S measurements.....	25
5.6	Load circuit measurements.....	27
5.7	Keypad and button.....	27
5.8	OLED functionality.....	28
5.9	RFID functionality	28
5.10	SD card	29
5.11	Menu functionality.....	29

5.12	Full system Test	30
6	Conclusions	30
6.1	Improvements.....	30
7	Appendix:	31
7.1	Appendix A:.....	31
7.2	Pin out configurations	31
8	References	33

List of Figures

1	Figure 1: Block diagram of the smart prepaid electricity meter	8
2	Figure 2: Power Regulator circuits (1 st figure), Thermal calculations (2 nd figure).....	9
3	Figure 3: Relay driven circuit.....	11
4	Figure 4: Circuit design of my op-amp circuit.....	11
5	Figure 5 Keypad Row and column configuration [12].....	12
6	Figure 6 Software flow diagram in main while loop.	14
7	Figure 7 S3 button bouncing flow diagram.	15
8	Figure 8 Keypad functionality.....	15
9	Figure 9 UART Flow Diagram for Receive function HAL_UART_RxCpltCallback:	16
10	Figure 10 Compare command flow diagram.	16
11	Figure 11 Send Student number on UART flow diagram	17
12	Figure 12 timing diagram	17
13	Figure 13 The flow diagram of the sample() function.....	18
14	Figure 14 Phase calculation flow diagram.....	18
15	Figure 15 ssd1306_conf.h file	19
16	Figure 16 Displaying multiple lines on OLED.....	19
17	Figure 17 FSM and interactions	20
18	Figure 18 State Diagram for OLED, menu transition, keypad keys.	21
19	Figure 19 Flow diagram of the Energy calculation.	21
20	Figure 20 append_log to log.csv file flow diagram.	22
21	Figure 21 Clear log.csv file.....	22
22	Figure 22 Dump function for log.csv.....	22
23	Figure 23 flow diagram of Last Units on Log.csv	23
24	Figure 24 Flow diagram of the RFID module interface.	23
25	Figure 25 Power supply test and test setup.....	24
26	Figure 26 start-up student number sent after correct time delay (160ms)..	24
27	Figure 27 The Rx pin receiving the “*Stat#\n” command from termite.	24
28	Figure 28 The output of my op-amp circuit for voltage, showing that the appropriate gain and offset is applied to the input signal.....	25
29	Figure 29 Calibration of voltage (Blue sinusoid) and current (Red sinusoid)	25
30	Figure 30 Phase measurements	26
31	Figure 31 The gate voltage when on (1st figure),coil voltage drop when on (2nd figure),gate voltage when off(4th figure), coil voltage drop when off(3rd figure).....	27
32	Figure 32 The S3 button bounce (1st figure), the keypad “1” key (2nd figure), Test setup for keypad(3rd figure).....	27
33	Figure 33 debugger test.....	28

34	Figure 34 SDA line pulled low (1st figure), Test setup (2nd figure), default display menu(3rd figure), specifications display page (4th figure).....	28
35	Figure 35 STM debugger and UART(1st figure),default page before units added(2nd figure),present tag page,(3rd figure),default page after units have been added(4th figure).....	28
36	Figure 36 Termite display (Left), Logging each second (Right)	29
37	Figure 37 Clipping Op-amp signal.....	30
38	Figure 38 Full SPEC schematic	31

List of Tables

39	Table 1 op-amp circuit test and comparisons to theoretical results	25
40	Table 2 Voltage, current and apparent power calculations.....	26
41	Table 3 Difference phase measurements(on the next page)	26
42	Table 4 Accuracy of the Real & reactive power.	27
43	Table 5 Menu functionality	30
44	Table 6 PIN out Table	33

List of Abbreviations

MCU – Microcontroller Unit (STM32G0B1RE)

SPEC - Smart prepaid Energy Controller

GPIO – General Purpose Input/Output

LED – Light Emitting Diode

NVIC – Nested Vector Interrupt Controller

I2C – Inter-integrated circuit

SPI – Serial Peripheral Interface

CS- Chip select

RST- Reset

MOSI- Master out slave in

MISO- Master in Slave out

SDA- Serial Data line

SCL- Serial Clock line

LED- Light emitting Diode

UID-Unique Identification

OLED- Organic light emitting Diode

CPOL- Clock polarity

CPHA- Clock Phase

MSB- Most significant Bit

RTC – Real-Time Clock

ADC – Analog-to-Digital Converter

HAL – Hardware Abstraction Layer

UART – Universal Asynchronous Receiver-Transmitter

DMA – Direct Memory Access

TIM – Timer Module

RMS – Root Mean Square

TRGO – Trigger Output

List of Symbols

ms- milliseconds

μ s - microseconds

° C/W- Degrees celius per watt

V- Voltage

V_{in}zs- input voltage zero scale

V_{in}fs- input signal full scale

V_{out}zs- output signal zero scal

V_{out}fs- output signal fullscale

V_{DD}- 3.3V

V_{SS} -0V(ground)

V_{ref}- stable voltage reference.

m- gain

b- offset

V_{in}- Input signal

1 Introduction

The Smart Prepaid Electricity Meter project addresses this need by providing a practical solution tailored to landlords, Airbnb hosts, and property owners seeking to monitor and control electricity consumption in multi-occupant settings such as student housing or short-term rentals.

This embedded system enables precise energy monitoring and load control. It functions as a standalone device that allows users to manage electricity usage via prepaid credit. Through an intuitive user interface—including a keypad and OLED display—the system provides real-time information on power consumption and available energy units. In addition, features such as RFID-based override access and SD card data logging ensure both usability and security for owners and temporary users.

Designed around the STM32G0B1RE microcontroller and built on a custom baseboard, the project combines analog signal processing, digital communication using UART on the STM32G0B1RE, and power electronics to realize a fully functional prototype. The system offers automated responses to high power usage and low energy thresholds, and supports emergency manual override, allowing the owner to use it in emergencies.

2 System description

2.1 System overview

The system is built around an STM32G0B1RE microcontroller, which serves as the main controller for both measurement and control tasks. It runs off a 9 V DC power source, which is connected to a regulator on the input of the board to produce stable 5 V and 3.3 V supply lines. A simplified block diagram of the system is shown in Figure 1.

The STM32 is connected to several key components, including an op-amp-based signal conditioning circuit, a relay driver for switching the load, debug LEDs, Keypad for user input, OLED Display for menu transition, Emergency Load control switch, RFID for authentication, SD-card module for logging important data logging and a UART interface used to communicate with the test station.

2.2 System requirements

The system requires monitoring and management of electricity consumption, The SPEC needs to integrate functionality to regulate external input power, measurement of V/I, control, and communication. The system operates from a 9V DC power supply and generates stable 5V and 3.3V supply rails within a $\pm 5\%$ tolerance to supply stable voltage to external peripherals. It must accurately measure and compute energy metrics, including voltage, current, phase, and power, from analogue signals within a 0–3.3V range without signal saturation. Measurements are displayed on an OLED display in real-time and reported via UART after specific commands are received like *Stat*, *Log* and *Dump* commands.

A user-friendly interface is achieved through integrating a 4×3 keypad for navigating OLED display pages, load toggling, and activating of energy metering. Secure RFID authentication is required for critical actions, such as adding units and enabling energy metering, which ensures controlled prepaid credit management. Load control is implemented via a relay as well as multiple debug LEDs to indicate load state (D2), low units (D3), energy usage (D4), and high power events

exceeding 3000W (D5). An LED also simulates the external load for visual verification. An emergency override S3 button with debounce handling allows control to turn on/off the load.

All relevant power measurements, timestamps, and units is logged to an SD card to enable historical tracking and is retained across power cycles. A Real-Time Clock (RTC) module ensures continuous and accurate timekeeping.

3 Hardware design and implementation

This section describes the hardware design and choices made for the SPEC. Each section has a design motivation, calculation, and justification as per the report rubric.

3.1 Hardware Block Diagram and Description of Interaction

The following figure describes a modular block diagram for the hardware description of the smart prepaid Electric meter.

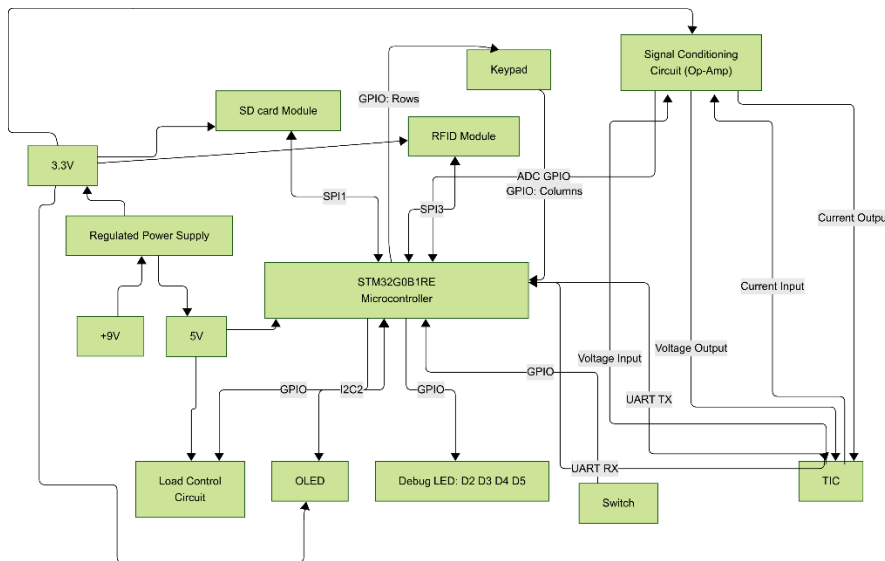


Figure 1: Block diagram of the smart prepaid electricity meter

Block diagram interaction:

- Power supply provides stable 5V and 3.3V from a 9V DC source.
 - 5V power for relay and STM32G0B1RE MCU.
 - 3.3V powers SD-card module, RFID module, both Op-amp circuits, OLED display
- STM32G0B1RE MCU manages:
 - the ADC sampling, UART communication with the Test station, relay control, keypad input and OLED output, RFID input and SD-card logging.
 - Internally: Timing of ADC and RTC
- The operational amplifiers:
 - applies positive gain and offset to the input signal from the TIC so that it is within the range of the ADC input for both voltage and current input signals.
 - A 2Vpp signal simulates 230Vrms mains voltage and 15Arms mains current
- Load Control circuit:
 - contains a J107 relay and an output LED, triggered by a 2N7000 MOSFET circuit from a GPIO PIN.
 - Turn on Load simulating LED.
- The emergency override button and keypad:
 - Keypad enables the user to navigate to different display pages as well as add units
 - to activate and deactivate the LOAD in case of emergencies.
- The OLED display:
 - Displays certain menu pages depending on which key sequence is pressed.

- Units can be added as well as turning on or off the load.
- UART Test interface:
 - Messages allow for external command and monitoring using a serial connection.
 - Sends status report of energy metrics like V,I,P etc.
 - Enable SD-card to log or clear file.
- SD-card module:
 - Enables the SPEC to store valuable energy metrics such as the date, time, V, I, phase and the units left.
 - Remember important Units information when there is a power outage.
- RFID module:
 - For security purposes, when adding pre-paid units.
 - turning off energy metering.
- Debug LEDS:
 - Allows the user to visually see high energy warnings, low units, and wh decrement without OLED display.
 - D2 load simulating circuit(Off/On)
 - D3 flashes when Units <min units.
 - D4 Flashes with each Unit decrementing (Wh)
 - D5 Flashes for high Real power>3000W

3.2 Power Supply

The SPEC is powered by a 9V DC power supply however, the power needs to be stepped down to stable 5V and 3.3V to be used by peripherals on the in the SPEC.

5V power Regulator:

- The D1 diode is to ensure that the signal the regulator receives is only DC. It also protects the rest of the circuit in case of reverse polarity power connection.
- L7805CV(TO-220 package) Regulator [13].
- The input decoupling capacitor C1, 100nF, is used to suppress high-frequency input noise and voltage spikes.
- The 100nF output capacitor enhances transient response as well as reduces ripple voltage under Load change conditions.
- The 10 μ F capacitor is used to further stabilize the output of the 5V regulator further so that the 3.3V regulator has a more stable input and does not need an input Capacitor
- No need for Heat Sink as maximum junction temperature is 105°C.

3.3V power Regulator:

- L78L33 (TO-92 package) Regulator [14].
- The 100nF capacitor ensures stable and low noise output for the 3.3V rail.

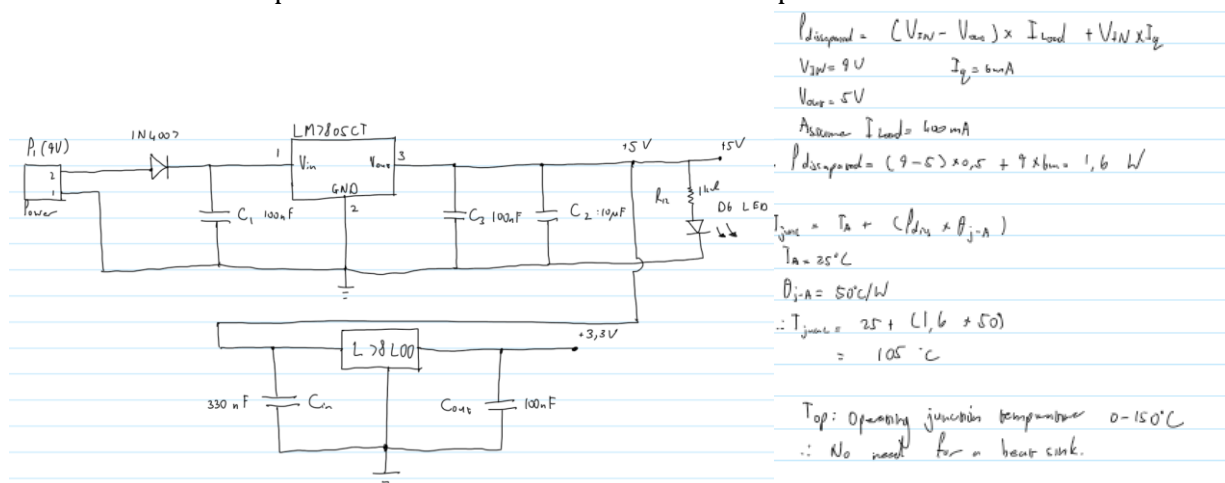


Figure 2: Power Regulator circuits (1st figure), Thermal calculations (2nd figure).

3.3 LEDs (Debug):

LEDs are used for Debugging purposes as well as for physical feedback that a circuit is working as intended. All LED's turn-on voltage is 2V [16].

- D6 LED:
 - Connected from the 5V supplied from the voltage regulator to a 1kΩ resistor to ground.

D2-D5 LED:

- D2: Indicates if the Load is switched on, Driven with PA10 which has no other special functionality.
- D3: Indicates that the energy units are low and flashes every 500ms. Driven with PB5.
- D4: Flashes each time one Wh unit is consumed. Will flash every second if Real power is above 3600W.
- D5: Flashes if the tenant uses more than 3000W of Power. Warning tenant of high power usage.
 - Drive the LED using PA10. Pin was chosen as it does not have any special functionality.
 - GPIO Pin output is 1.62V-3.6V [9].
 - Pin has an operating output of 8mA [9].
 - Toggles in response to “*Load#\n” command.

A 270-ohm resistor was chosen to limit the current through the LED.

$$I = \frac{3.3 - 2}{270} \cong 4.81mA$$

This ensures that the LED does not exceed the 8mA current limit from the MCU GPIO PIN as well as being bright enough to visually see in bright settings.

3.4 Relay and control:

The STM32G0B1RE's Relay control allows the user to turn on bigger external loads with a GPIO signal from an MCU.

Design Considerations:

- Using a 2N7000 N-MOSFET to switch bigger loads using a smaller voltage i.e GPIO PIN.
- Used a 10kΩ external pull-down resistor to ensure the PIN does not leave the gate floating and biasing the Gate to the NMOS with a Voltage greater than 3V [18].
- The J107 Series 1A relay operates with any voltage larger than 3.75V [15].
- Include a flyback diode on the relay's coil nodes to decrease voltage spikes or reverse EMF from the relay which can damage the NMOS.

Circuit Design:

- Configure PA10 GPIO as Output, 3.3V high [9].
- Connect a 270Ω series resistor to mitigate inrush current from the GPIO, which could damage the driving circuit.
- Connect the source of the MOSFET to ground as well as a 10kΩ pull-down resistor from the Gate of the MOSFET to ground.
- Power the Relay coil with one of the 5V rail voltages.
- Connect a Flyback diode, 1N4148, across the relay coil.

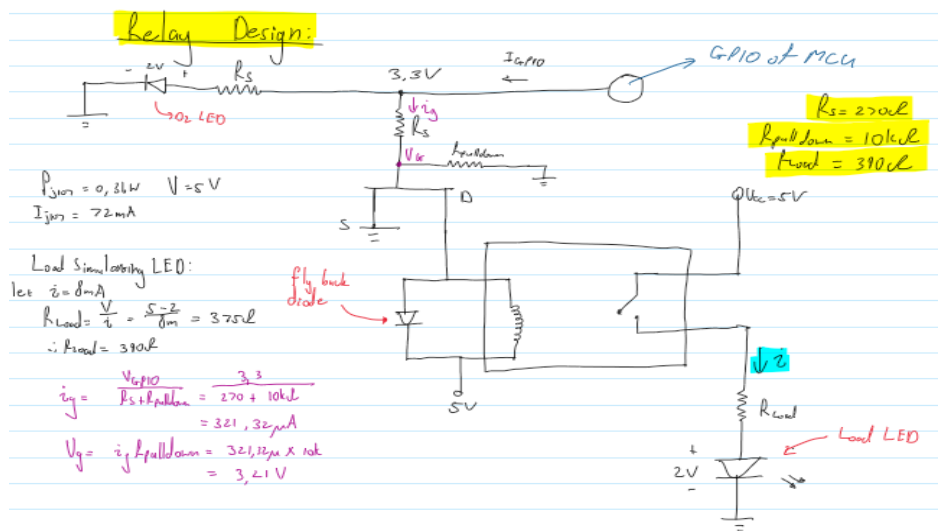


Figure 3: Relay driven circuit

To summarize when PA10 is driven high. The gate Voltage of the MOSFET exceeds the gate threshold thus turning on the MOSFET. This allows current to flow through the relay coil and the relay is turned on.

3.5 Op-amp configuration and design

Need to design an op-amp circuit to apply the correct gain and offset to a 2V peak-to-peak input signal. The input to the ADC (analog-to-digital converter) on the STM32 board only takes positive voltages from 0V to 3.3V [9]. The Opamp takes a signal which is centered around 0, shift's it upward and centers the signal around 1.65V, also by applying a gain to improve the resolution of the signal. We can then sample the wave uniformly to reconstruct it and do calculations with this data to get the correct RMS voltage.

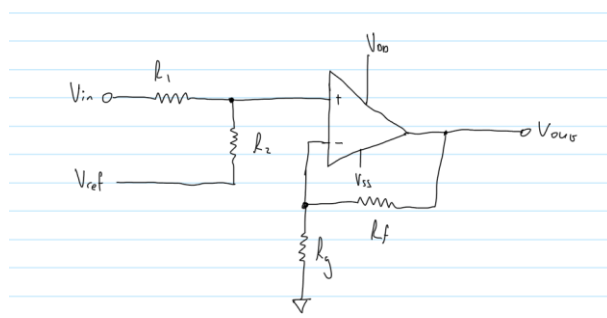


Figure 4: Circuit design of my op-amp circuit.

The Op-amp chosen for this design is the MCP602, a dual Op-amp [5]. The reason for using a dual op-amp is for the case of applying the gain and offset to the voltage and current. Designing the operational amplifier circuit using (SLOA097)“Designing Gain and Offset in 30sec” PDF [6]

The above diagram of the non-inverting operational amplifier is for the case of positive gain and positive offset

Input voltage range - The min (middle of the sine wave) (V_{inzs}) and max (V_{infs}) of your input signal

Output voltage range - The min (middle of the sine wave) (V_{outzs}) and max (V_{outfs}) voltage for output

V_{DD} : 3.3V will cause the op-amp to saturate at 3.3V, which will minimize the voltage greater than 3.3V into the ADC. It acts as a safeguard for the ADC.

V_{SS} : 0V(ground)

V_{ref} : 3.3V was chosen as it is a convenient stable reference, since the V_{DD} is also 3.3V. It gives an appropriate bias to create the needed offset.

V_{infs} : 1V; V_{inzs} : 0V; V_{outfs} : 3V; V_{outzs} : 1.65V

V_{in} : This is the 50 Hz 2Vpp input sinusoid to which the gain and offset must be applied.

m: gain

b: offset

$$m = \frac{V_{outfs} - V_{outzs}}{V_{infs} - V_{inzs}} = \frac{3 - 1.65}{1} = 1.35 \quad (eq. 1)$$

$$b = V_{outzs} - m \times V_{inzs} = 1.65 - 1.35 \times (0) = 1.65 \quad (eq. 2)$$

Choice of resistors:

Choose standard resistors for both R1 and Rf. On the pdf it said to use a specific datasheet, which couldn't be found, to choose the correct Rf. However, The Design worked out perfectly as the circuit was able to apply the correct gain and offset.

R1: 10kΩ

Rf: 47kΩ

$$R2 = \frac{V_{ref} \times R1 \times m}{b} = \frac{3.3 \times 10k \times 1.35}{1.65} = 27k\Omega \quad (eq. 3)$$

$$Rg = \frac{R2 \times Rf}{m \times (R1 + R2) - R2} = \frac{27k \times 47k}{1.35 \times (37k) - 27k} \approx 56k\Omega \quad (eq. 4)$$

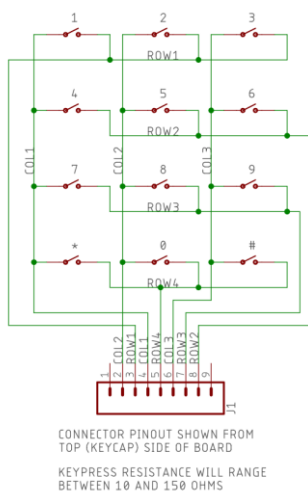
The output voltage waveform and current waveform is connected to PA0 and PA1 respectfully.

3.6 Emergency Load Button

Using an External switch, the owner can turn off or On the load. This allows for testing and as a fail-safe. The Button will be used in an active low State. This means that it pulls the GPIO PIN to ground and triggers on a Falling edge. Connect one of the button's legs to the chosen GPIO PIN and the other leg to ground. The button connection has a internal Pull-up resistor to ensure Pin is not floating.

3.7 Keypad

The Keypad provided to us is 4x3 matrix of rows and columns. Using this matrix, only 7-GPIO Pins are needed to read all 12 keys on the keypad. Using a Keypad document which one of the Students have previously designed to visually depict which keypad connection Pin relates to the rows and which relate to the columns.



Design considerations:

- The keypad does not need any external power source as It makes a connection between a high which is high and a column that reads this high value.

Pin configurations:

- Using the UM2324-STM32 the following Pins were selected for the Rows and configured as GPIO output: PC1, PB10, PB13, PB15.
- The following Pins were selected for the columns and configured as GPIO input. Enabling the External interrupt for further efficiency: PC0, PB2, PB6.
- The reason for these pin selections is due to the User manual. The pins are depicted as General-Purpose Input and Output and do not have any special functions i.e, I2C or SPI

Figure 5 Keypad Row and column configuration [12]

3.8 OLED

The OLED Used in the project is the SSD1306 display with a 128x32 resolution, input voltage of 2.5V-5.5V, stated in the PDD[7]. The communication scheme used for this peripheral is I2C. There are 4 Pins on the Board.

Circuit Design:

- Using the input voltage range, a stable 3.3V voltage is selected as the power source for the OLED display.
- The Two Pins needed to connect to the MCU is the SDA line and the SCL line.
- The SCL Pin chosen is PA11. This pin carries the clock signal and synchronises the data transfer.
- Can only write data when SCL goes low, as it is defined as an active low configuration. The STM board controls the SCL line. It can request or send data to the OLED.
- The SDA line Pin chosen is PA12. This pin carries the Data for both the MCU and the OLED.
- The SDA and SCL PIN need to be configured as GPIO output is high with a Pull up resistor in the IOC [10].

3.9 SD Card

The SD card's purpose is to log Power data. This enables the owner to track the tenant's energy usage.

Circuit Design:

- The SD card is powered with a 3.3V supply from the 3.3V rail. This is within the SD-card specified range of 2.6-3.6V as stated in PDD [7].
- There are 6 pins on the SD-card. The communication scheme used is SPI1.
- GPIO Pin configuration:
 - CS is connected to PC8, and the state is high. A internal pull-up resistor is used to ensure the SD card is not accidentally triggered [1].
 - PD8 used for CLK pin, has a internal pull-up resistor for additional stability [1].
 - PA6 and PA7 are used for MOSI and MISO, respectively. Also, have internal pull-up resistor enabled [1].

3.10 RFID reader (MFRC522)

The RFID reader is used as security measure to enable or disable energy metering, depending on the owner's needs, and to top up units for the tenants who have bought.

Circuit Design:

- There are 7 Pin connections on the RFID module [17].
- The RFID is powered by 3.3V supply from the rail.
- The Reset Pin and SDA (CS) Pin are connected to PD2 and PB8, respectively. Both have internal pull-up resistors [4].
- The CLK, MISO, and MOSI Pins are connected to PB3, PC11, and PC12, respectively. These pins are the SPI3 pins. Each Pin has a internal Pull-up resistor for extra stability during communication [4].
- A 100nF Capacitor is added across ground and 3.3V as to mitigate the EM noise on the OP-AMP created by the relay and the RFID Module.

4 Software design and implementation

4.1 Software Block diagram and description of interaction

At the Start of the program all the peripherals are initialized and ready for communications. The Drivers and different peripherals enable the system to coordinate UART communication, ADC sampling,

RFID polling, SD-card logging, Energy metering and LEDs to turn on all in perfect timing. The system mainly checks for flags in the main loop which link to other functions that either complete in the back end or raise interrupts for the CPU to intervene. The key functionality can be seen in the flow diagram below.

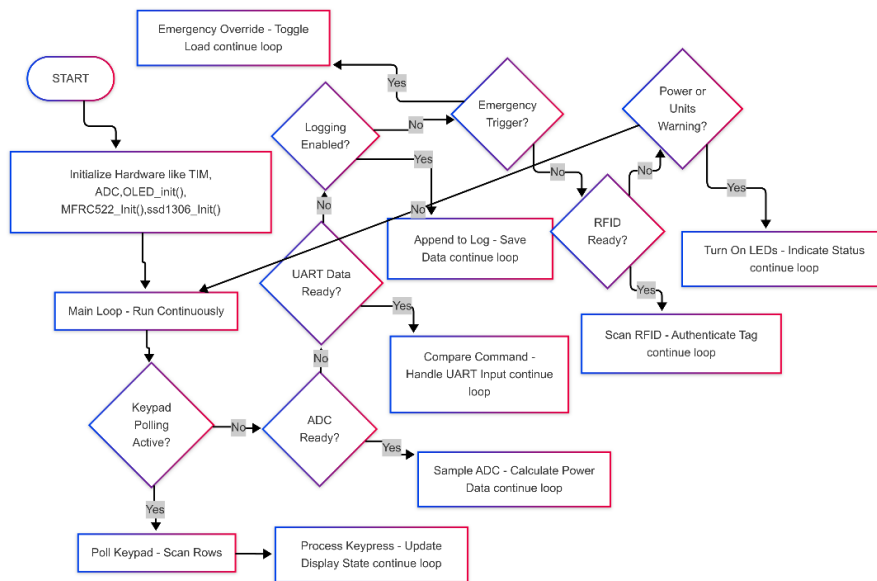


Figure 6 Software flow diagram in main while loop.

- **Keypad polling:** checks if the correct key is pressed and is debounced for 50ms.
- **ADC sampling:** Checks if the ADC has completed sampling the waveform and added the data to the respective buffers. Calls the sample() function to calculate the related power calculations. This occurs every time the ADC is finished sampling.
- **UART:** Checks if the UART messages has been received and compares the command to predetermined commands for certain operations to take place. E.g “*Stat#\n” To display all the current power metrics
- **SD-card logging:** Checks if the SD card needs to log valuable information about the systems power usage every second.
- **Emergency trigger:** Checks if the S3 button has been pushed to turn on the load simulating LED or to turn it off.
- **RFID:** Checks if the system STATE is in RFID state after commands like “*Unitsxxxxx#\n” or if the Energy count wants to be enabled/disabled.
- **Power warning, Units:** Relate to the Debug LEDS and when they need to turn on. D3 blinks to indicate low units. D4 blinks once per 1Wh of energy is consumed. D5 blinks if power is greater than 3000W.

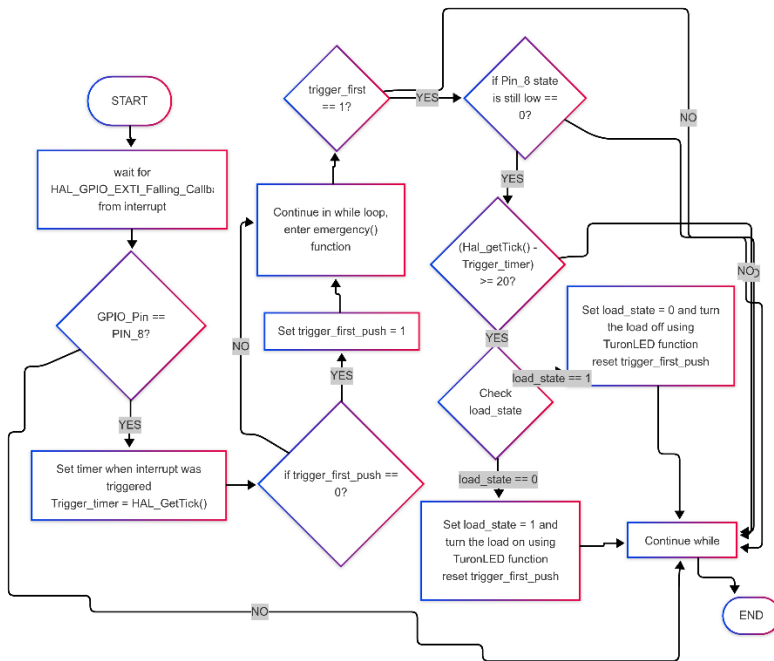
4.2 Button bounce handling and emergency S3 button

(Both the keypad as well as the S3 emergency button have the same button bouncing handling logic.)

The S3 button is configured as an active low configuration, which means that the button will connect the output pin to ground and pull it down.

- GPIO Pin used was PA8 as it has not special functionality other than IO
- Configure PA8 as External interrupt mode with falling edge detection.
- Enable an internal Pull-up resistor to always ensure the pin is high when nothing is connected to the input.
- The Load should not turn on unless there are units available on the system. It should turn the load off if the load is on, vice versa.

HAL_GPIO_EXTI_Falling_Callback (Interrupt Trigger). This function is called when a falling edge is detected on GPIO pin PA8, which is configured as an interrupt input for the emergency button.



When the user clicks the button falling edge is created and the event it triggered. In the HAL_GPIO_EXTI_Falling_Callback() function we check to see if the correct PIN caused the interrupt. In this case PA8. Set a timer variable, trigger time, which will be used to keep track that the button is held down long enough. Set a flag in-order for the program to step into the functionality to handle the bouncing in the emergency function.

In the emergency() function check if the interrupt was triggered. If so get the current time of when entering the function. If the Pin is still low and the current time subtracted by the trigger timer is larger than 20ms, then using the Load's current state, either turn the load on or off.

Figure 7 S3 button bouncing flow diagram.

I set up a 2D array of all the keys on the keypad and assigned numbers 0-9 for all the numbered keys and 10 and 11 for the '*' key and '#' key, respectively. Using this 2D array the algorithm can pinpoint which key was pressed but adding the active row and col_pressed variable into the keypad_map array.

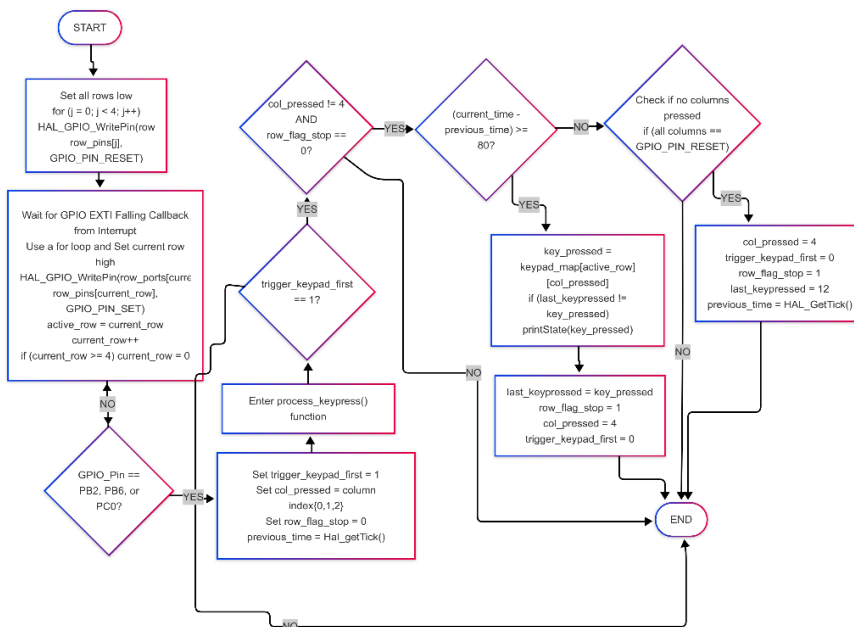
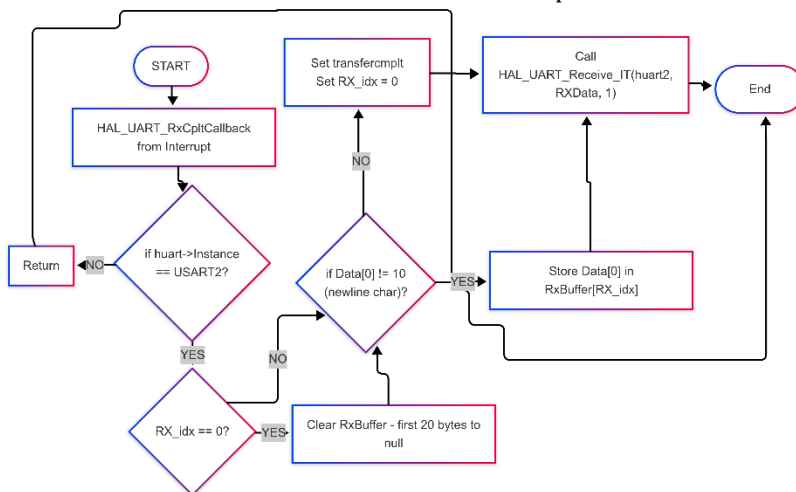


Figure 8 Keypad functionality

4.3 UART communications (protocol and timing):

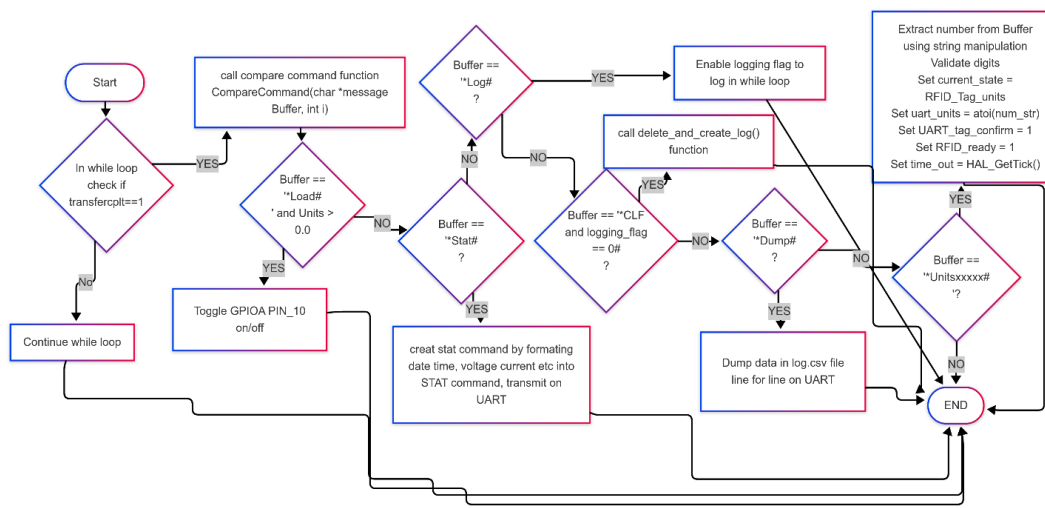
UART Setup:

- Baud Rate: 57600 bits/s
- On the IOC under connectivity enable USART2 as Asynchronous.
- Word length should be 9 bits including parity.
- Enable the NVIC settings for USART2.
- Data Format 11 bits: 8 data bits, Even Parity, 1 stop bit.
- Time to send 1 byte: $1 \text{ byte} = \frac{11}{57600} = 191\mu\text{s}$
- Commands to trigger UART:
 - `*Load#\n`: Have no UART Response but toggles D2 LED as well as Relay circuit
 - `*Stat#\n`: Calls a function to Calculate Voltage, phase, current, Reactive Power, Real Power, and time. Return all this data to the Test Station RX pin.
 - `*Log#\n`: Starts the SD-card to log data
 - `*CLF#\n`: Clears the log file of all the data and creates a new one
 - `*Dump#\n`: Dumps all the logged data line by line over UART
 - `*Unitsxxxx#\n`: Adds units to the system for the tenant to use.
- Use UART receive function in Interrupt mode and use UART in Normal transmit mode.



Using UART in interrupt mode we append 1 incoming data byte to the buffer called Data[0] calling the following command HAL_UART_Receive_IT(&huart2, RxData, 1). The byte in Data [0] is added to the global character array RxBuffer, which is used to store the full command received on UART. A flag, transfercplt, is set once the complete command is sent. In the Main while loop, we handle the flag so that we can see what process needs to execute depending on the command that is received. Using the compare command function.

Figure 9 UART Flow Diagram for Receive function HAL_UART_RxCpltCallback:



The above figure shows the flow of how each UART command gets processed as it is received. Some of these commands either display responses back over UART or interact with the SD-card to either remove or add data.

Figure 10 Compare command flow diagram.

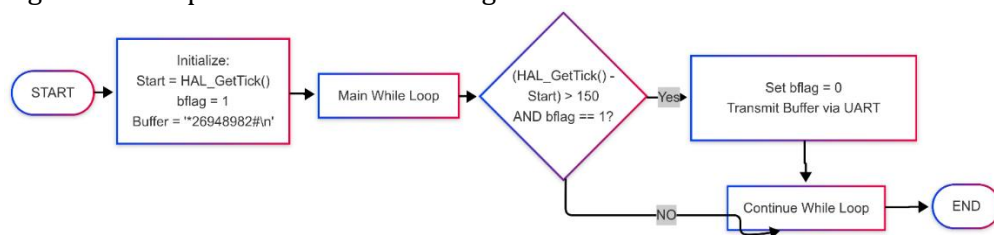


Figure 11 Send Student number on UART flow diagram

Will transmit the student number via UART after a 150ms delay after power-up.

4.4 ADC, Setup, Calibration, and Processing

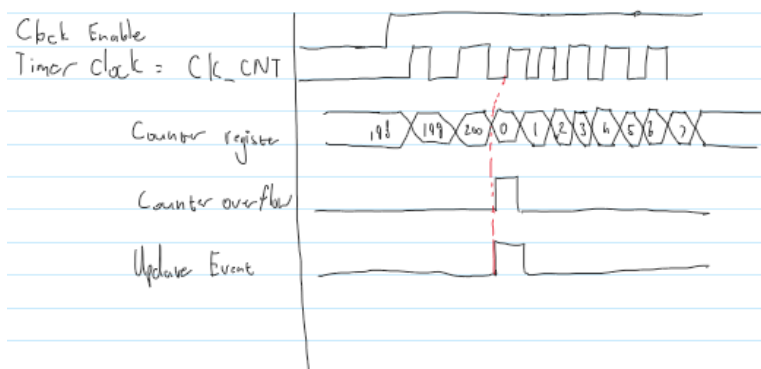
Sample a waveform of the output signal from the Op-amp into the ADC. Sample the wave using the ADC in DMA mode. Trigger the DMA to sample using a timer. The 12-bit ADC clock timer is set to 64 MHz. The frequency of the input sinusoid is 50Hz and the maximum amplitude is 2.2Vpp. It has a period of 20ms. To have 100 samples of each wave. The timer that activates the DMA to sample the ADC needs to overflow every 200 microseconds.

Timer 2 Configuration:

Set-up general purpose timer TIM2 to trigger the ADC to sample using DMA. TIM2 uses the internal clock, which is set to 64MHz.

- Set the prescaler of the timer to 63, This will cause the timer to tick every 1µs.
- The auto-reload value(ARR) should be set to 199 so that the timer triggers an Update event every 200µs. This update event will trigger the ADC to sample at exactly 200µs if the ADC is set to start when and TRGO(trigger output event) has occurred.

$$f_{timer} = \frac{f_{clk}}{prescaler} = \frac{64Mhz}{63 + 1} = 1Mhz$$



The following timing diagram describes how the timer creates an update event to trigger the ADC to start sampling the current value it has. The ADC is setup with the following parameter settings. It is a 12-bit ADC with a sampling time of 19.5 clock cycles. The system clock frequency is 64MHz, which has a clock cycle of 15.6 nanoseconds. The time needed to sample the value from the ADC into the memory buffer is 19,5 x 15.6 ns = 304.6 ns. This means that from when the update event is triggered until the DMA can be used again to sample the values from the ADC to the next buffer position is 305ns. Choosing 19.5 cycles allows enough time for the capacitor to charge and transient effects to be minimised before extracting the value and getting good reading.

Figure 12 timing diagram

The following flow diagram is how I programmed my algorithm for capturing the Rms voltage from the input signal that the ADC reads.

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    /*Prevent unused argument(s) compilation warning*/
    UNUSED(hadc);
    //Function is like a flag to signify that the ADC has sampled a value and
    //allocated it to the correct buffer.
    //Process works independently of CPU
    adc_ready = 1;
    HAL_ADC_Stop_DMA(*hadc1);
}
```

Figure 4.4.2: Conversion complete function used to change adc_ready variable.

As the board is turned on, global variables are initialised in the main function, and the TIM2 is started indefinitely. The ADC starts sampling in DMA mode and fills ADC_buff[200] array. After the timer has triggered an updated event for the ADC to sample, A Conversion complete callback is generated from the DMA once the values are in the ADC_buff that the DMA read into, and then the ADC-ready flag is set. The sample function is called. The interleaved buffer voltage and current values are extracted from the ADC_buff into sperate voltage and current arrays. convert each value from ADC_buff to normal voltage before you add it to the separate arrays. Each value is in the range 0-3300 mV, since the ADC samples the values with a number between 0-4095.

$$V = \frac{ADC_buff[2 * i + 1] \times 3.3}{4095} \quad C = \frac{ADC_buff[2 * i] \times 3.3}{4095} \quad \text{where } 0 \leq i < 100$$

Find the minimum and maximum voltages with a for loop to find the peak-to-peak voltage. Knowing that 2Vpp is equivalent to 230V rms. After getting the pk-pk voltage, calculate the Real Voltage using the following equation.

$$V_{rms} = \frac{Pk\text{voltage}}{1000 \times 2 \times m} \times 230 \quad A_{rms} = \frac{Pk\text{voltage}}{1000 \times 2 \times m} \times 15000$$

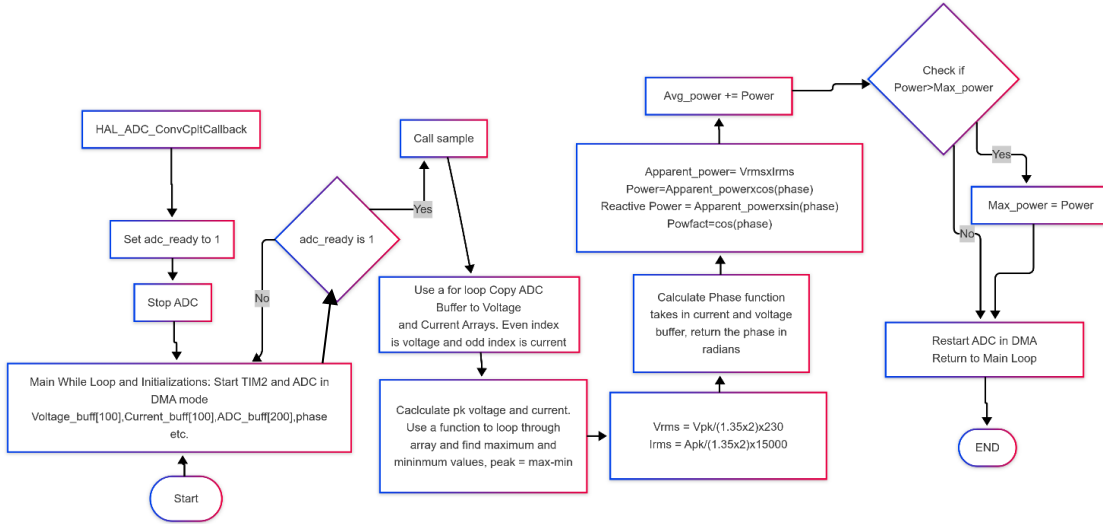


Figure 13 The flow diagram of the sample() function

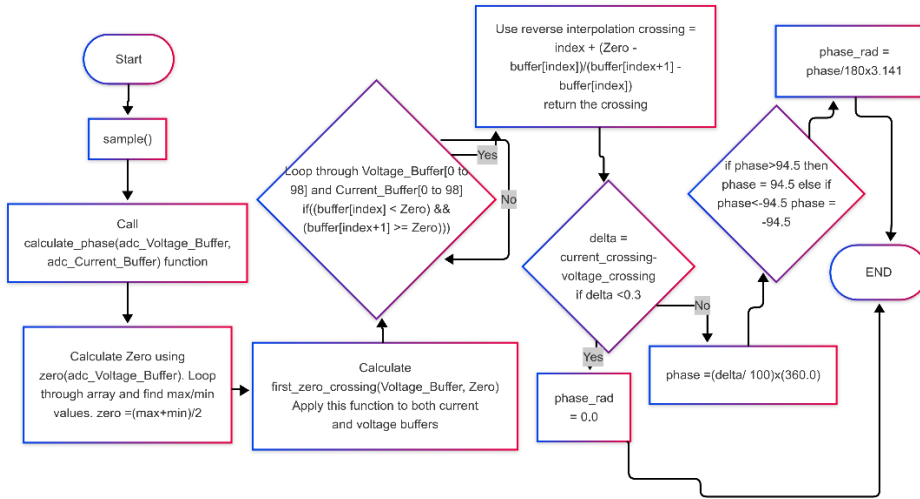


Figure 14 Phase calculation flow diagram

To calculate the phase of the two input signals, I first had to find the first zero-crossing of each wave. Use reverse interpolation to find the most accurate zero location, and then subtract the zeros from each other and multiply by $\frac{2\pi}{100}$ to get the phase in radians. Using this, we can calculate the real and reactive power.

$$\text{power.crossing} = x1 + \frac{(x2-x1)}{(y2-y1)} \times (zero - y1)$$

4.5 OLED

The OLED used is based on the graphic driver SSD1306. This section will describe setting up the driver files and the IOC.

IOC Setup and Drivers [10] & [11]:

- Copy the driver files with the '.h' file extension into the Inc folder inside the core folder of your project.
- Copy the driver files with the '.c' file extension into the Src folder inside the core folder of your project.

- Go into the `ssd1306_conf_template.h` file to set up the OLED so that the MCU recognizes it when you try and use the functions from the `ssd1306.c` file.
- In figure 10 below you need to comment on all the STM controllers defined except the STM32G0
- Comment out or delete all the SPI configurations which is defined and only keep interface type I2C.
- In your IOC under connectivity, scroll down to I2C2 and enable it to I2C in the mode tab.
- Change the I2C speed Frequency tab to 100kHz under parameter settings.

Timer 3 was used to update the OLED display every 200ms. The timer was set up in interrupt mode. The following settings were toggled in the ioc to allow the timer to trigger every 200ms.

Setup Timer 3:

- In the clock configuration. The APB speed, which determines the frequency, is set to 64MHz.
- In the TIM3's parameter settings set the prescaler to 63000. To set the frequency of the clock to 1000Hz.
- Set the auto reload value to 199 and the trigger output (TRGO) to Update Event.

```
// MCU Family selection
#define STM32G0 // You are using STM32G0B1RE

// Select interface type
#define SSD1306_USE_I2C // Use I2C interface
// #define SSD1306_USE_SPI // Uncomment to use SPI instead

// I2C configuration
#define SSD1306_I2C_PORT hi2c2 // I2C handle (PA11 = SCL, PA12 = SDA) using ti
#define SSD1306_I2C_ADDR (0x3C << 1) // SSD1306 7-bit I2C address shifted for HAL

// Display resolution
#define SSD1306_WIDTH 128 // Width of the OLED in pixels
#define SSD1306_HEIGHT 32 // Height of the OLED in pixels
```

Figure 15 `ssd1306_conf.h` file

Change the defined port from `hi2c1` to `hi2c2`, as that is the I2C configured on the MCU. Define the width and height to 128 and 32 bits, respectively as this is the width and height of our OLED device. After this, rename the file to `ssd1306_conf.h`

On powerup, I start the timer 3 in interrupt mode and then use the **void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)** function in my main function. I update a flag, which in my main while loop, I check to see if the update flag has been set and refreshes the OLED with `printState`.

A function I created to display multiple lines on the OLED. The reason for this function is to format each line on the display to 20 characters long. Since the display is 128x32. I chose to use the Font 6x8. Each Character is 6-pixels wide and 8-pixels tall. This means that one line can be $\frac{128}{6} \approx 21$ characters long. Up to 4 Rows of 21-character-long strings can fit on the display. In the display multiple string's function, I formatted each string that it received from the Print State command so that the OLED display would match what was seen on the PDD. The format of a example string is as follows: "Apparent:%8dVA\nEnergy/d:%8dWh\nUnits:%10.3fkWh\n".

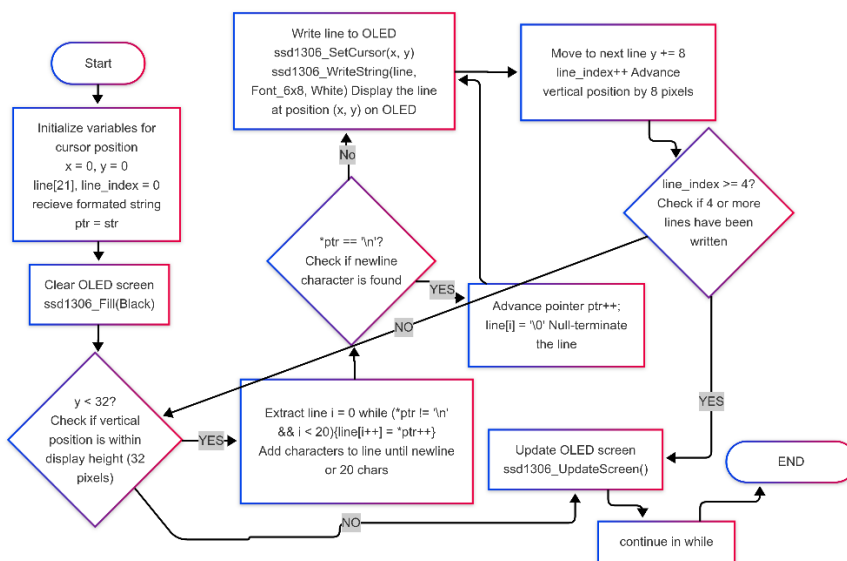


Figure 16 Displaying multiple lines on OLED

Each line is built with a while loop. The string is included as a parameter. If the index position is not greater than 20 characters and the line does not have a '\n' character. If this is true, I null terminate the line. Using the following function `ssd1306_SetCursor(x, y); ssd1306_WriteString(line, Font_6x8, White);` I can write the line to the OLED buffer. Increment the y-position to fit the next line and start the process over. Once all the lines are added to the buffer I update the screen.

4.6 FSM

The FSM of interaction of all the subsequent steps. The program starts by initializing all the peripherals as well as getting previous Units from SD-card, starting OLED on the default page etc... The program steps into a infinite while loop which checks for flags and waits for interrupts to call critical functions which alter the State.

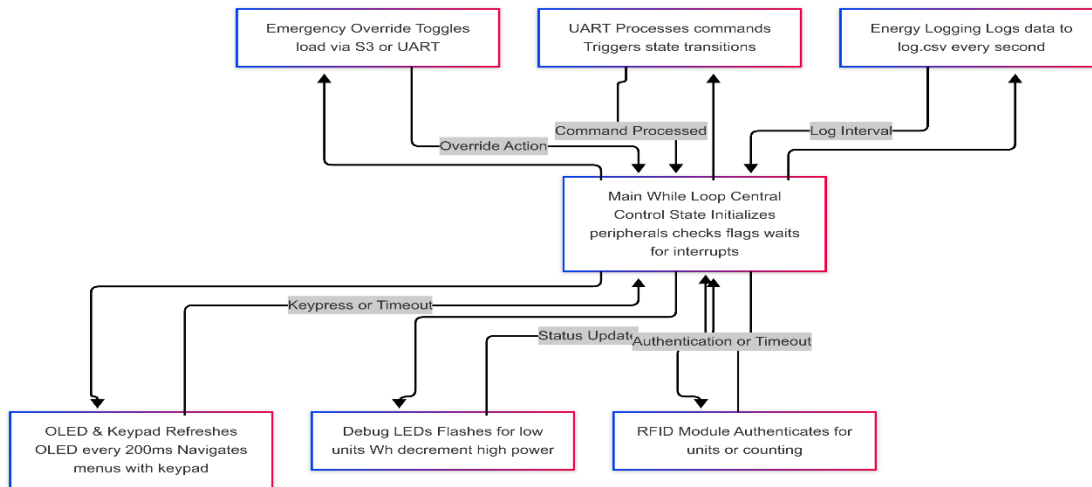


Figure 17 FSM and interactions

OLED and Keypad:

The above diagram is the State diagram of the OLED and which key was pressed. The system refreshes the OLED display every 200ms which will allow the OLED to update In real time. The PrintState() function handles the state transitions and the display function, [Figure 16](#), do format the OLED output.

UART:

From any of the above states if one of the UART commands from [Figure 10](#) are received the correct state transition or data processing step will take place. E.g: Units command will go to the RFID_tag_units State.

Debug LEDs:

Used for analysis of the system without OLED. D3 indicates low units or No units. D4 blinks when 1Wh of Units is consumed. D5 LED toggles if the Real power exceeds 3000W.

Energy logging:

At any STATE When logging is enabled due to a UART command “*Log#\n” seen in [Figure 10](#). The program logs the correct data in the log.csv file at 1 second intervals as described in [figure 20](#). This includes the correct units which is shown in [Figure 19](#).

Emergency override:

In case of any emergency at any STATE in the FSM. The load simulating LED can be turned off/on, if units are available, using UART or S3 button at any time to simulate a override button as described in [Figure 10](#) and [figure 7](#).

RFID Module:

If a UART “*Unitsxxxxx#\n” is received, Energy count is enabled from STATE: unit count or STATE: add units on the OLED. The RFID module is activated and the OLED goes into RFID_tag_units or RFID_tag_count depending what the previous state was. In this STATE authentication is required in order to add the units otherwise nothing happens. Refer to [figure 18](#) or [Figure 10](#)

ADC sampling and calculations:

In any state and every while loop iteration the sample() function is called if the adc_ready flag was set after DMA completed. In this function the relative energy calculations take place like voltage, current, power, apparent, reactive, phase. Refer to [Figure 13](#) and [Figure 14](#).

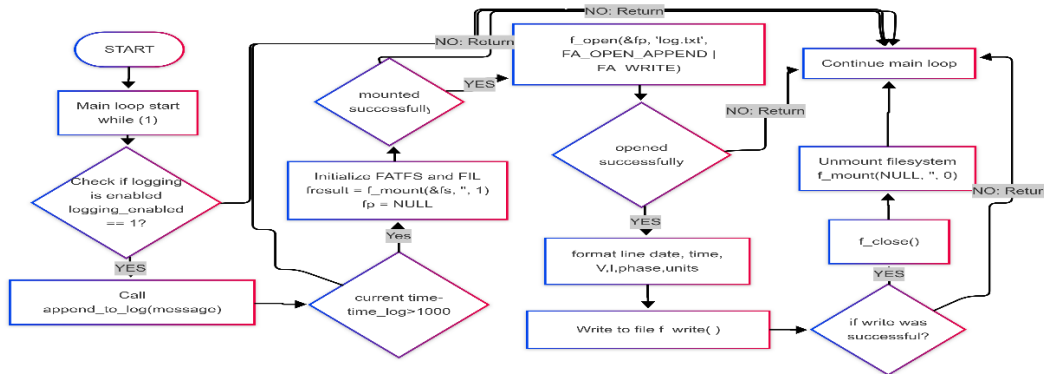
4.8 SD-card setup and Commands

Using the provided resource on the PDD from [7], I was able to integrate my SD card. The video tutorial [1] also gave a great understanding of how to set up the SD-card module in SPI mode.

In the IOC, select connectivity and SPI1. Select the Full-Duplex Master. Change the prescaler to 256 in the parameter settings for a 250 Kbit/s baud rate. Leave the rest of the settings as they are configured. The slower clock will improve any compatibility errors. Go to Middleware and software packages and select FATFs to interface with your SD card. The drivers used, git_repo [2], to interface with the SD card were the user_diskio_spi.c; however, for the setup, you need to change some settings in the user_diskio.c file.

All the below functions handle with a file named: log.csv

Append to log function():



This function writes a new line of data with the time, date, voltage, current, phase and Units to a csv file each second. This allowed for consistent energy consumption tracking. The following flow diagram explains the implementation of this function

Figure 20 append_log to log.csv file flow diagram.

Delete and create log function:

Erases the data within the log.csv file by creating a new file and deleting all the content in the old one.

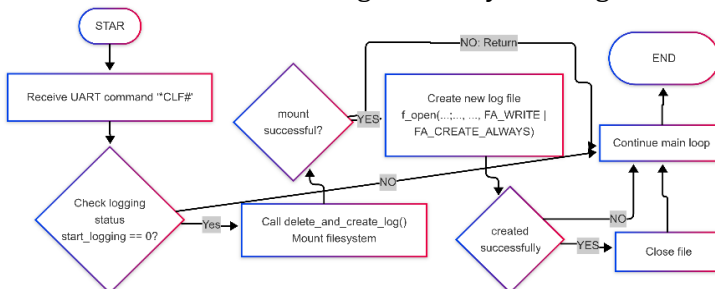
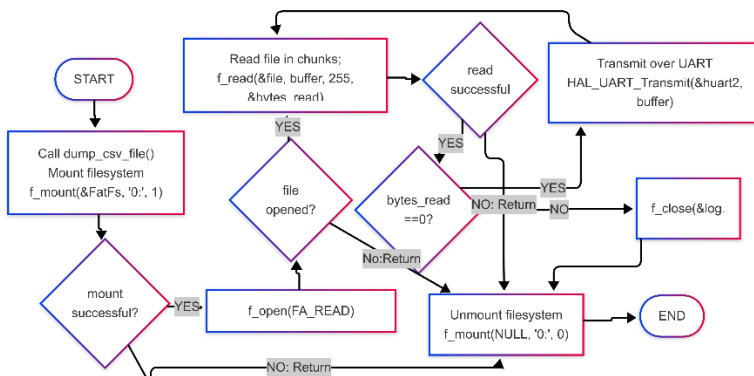


Figure 21 Clear log.csv file



Dump the contents of your log.csv file line by line over UART.

Figure 22 Dump function for log.csv

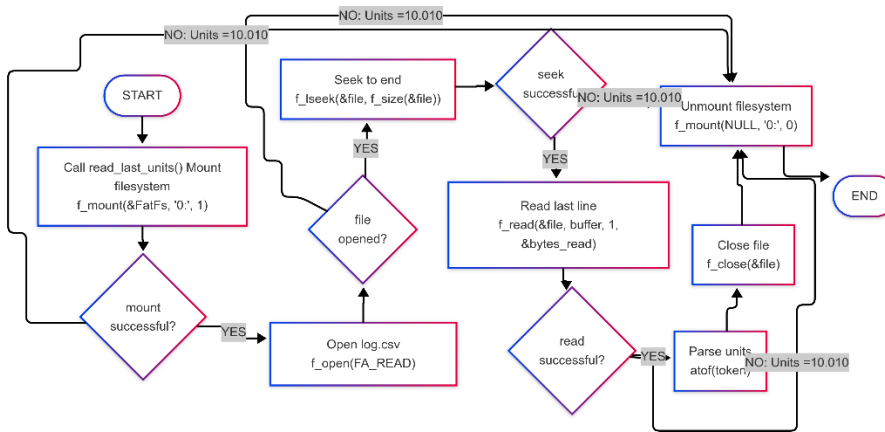


Figure 23 flow diagram of Last Units on Log.csv

4.9 RFID- interface

The RFID MFRC522 reader is connected via SPI3. Using the following guide on youtube [4] on how to connect and interface with the RFID module. The same as for the SD-card module, go into the IOC, under connectivity, click on SPI3, and enable mode as Full-Duplex master. Frame format: Motorola; Data size: 8-bits; first bit: MSB First. Prescaler = 256 for Baud rate of 256 Kbits/s; CPOL/CPHA set as 0/0, respectively. The main functions I used from the RFID drivers [3] are: MFRC522_Request(): to detect any RFID tags in the readers field, and MFRC522_Anticoll(): retrieves a presented tags UID.

The drivers RC552.h file's #include clause needs to be match with the current MCU connect. As well as the #define of the CS, MOSI, MISO, CLK, and RST pin need to be changed to match with PINS selected on your MCU. The following flow diagram describes how I implemented the RFID to authenticate adding Units and turning on Energy consumption.

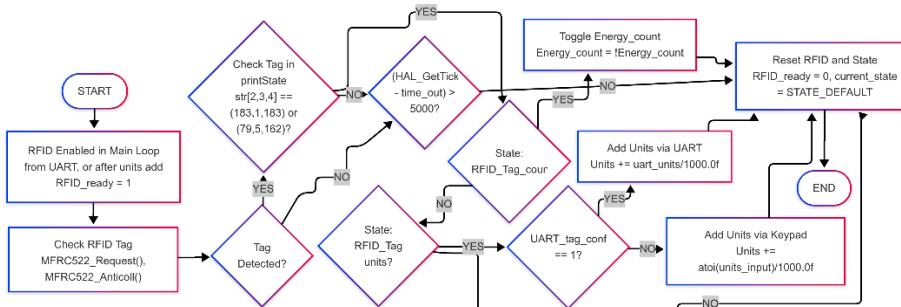


Figure 24 Flow diagram of the RFID module interface.

In figure 4.9.1, It is seen how the RFID module is utilised within the FSM. The only time the RFID is activated is after unit count is active, "*Unitsxxxxx#\n" command on UART or adding uits on the OLED.

5 Measurements and Results

The following measurements were taken of my SPEC to show that it complied with the specifications in the PDD as well as showing that the hardware and software aspect interact seamlessly and the program flows as intended.

5.1 Power supply measurements

The following measurements were taken of the power supply. The tests were taken as the DC-power supply voltage was varied to show that the 5V Regulator and 3.3V Regulator were working as per specifications. As seen in the figure as the DC-supply voltage is varied from 9.81V down to 6.33V we can see the output on the 5V and 3.3V regulators decreasing. Below 8V the regulator starts to show signs of being in cut-off stage as the output voltage on the regulators are below turn-on voltage. The percentage error of the regulator to depict if the powersupply is within 5% tolerance.

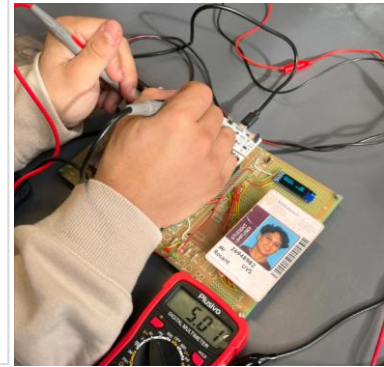
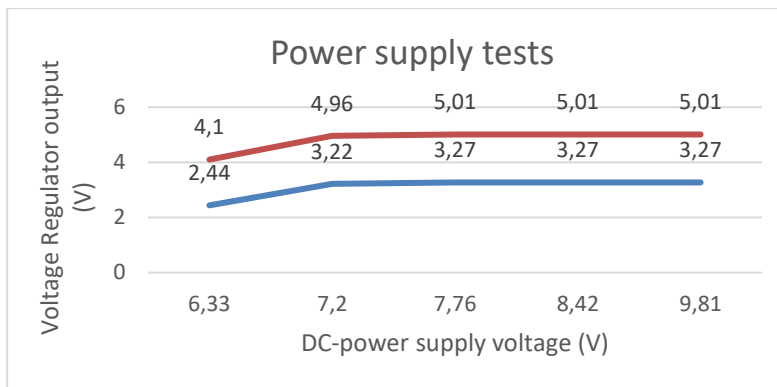


Figure 25 Power supply test and test setup

5.2 UART measurements

To interface with the board without using the keypad, you would use UART. Connected via serial communication to your laptop. Using software such as Termite to interface with your board. Verified the UART start-up time and student number being sent 150- 500ms after start-up on the TX pin (PA2). I also tested the “*Stat#\n” command being received by the RX pin (PA3).



Figure 26 start-up student number sent after correct time delay (160ms).



Figure 27 The Rx pin receiving the “*Stat#\n” command from termite.

5.3 Voltage and Current circuit Measurements

I tested my op-amp circuit as follows:

Offset:

- I connected a grounded signal to the input of my voltage op-amp circuit. Using this measurement, you can find the Offset applied by the Op-amp circuit.

Gain:

- Applying different peak-to-peak signals I was able to measure the gain my circuit applied to the signal and measure the error from the theoretical gain: 1.35.
- The figure below shows the test setup as well as the different input signals

Input Signal (V)	Output (V)	Offset (V) and Gain	Error %
0	1,72	1,65 V	4,24
1,16	1,4	1,2	11,11
2,16	2,76	1,278	5,93
2,4	3,04	1,271	5,93

Table 1 op-amp circuit test and comparisons to theoretical results

As seen in table the signal generator added a extra 10% voltage to the signal it was outputting. For the 1Vpp signal the output was 1.16Vpp on the oscilloscope showing a discrepancy of the signal generator.

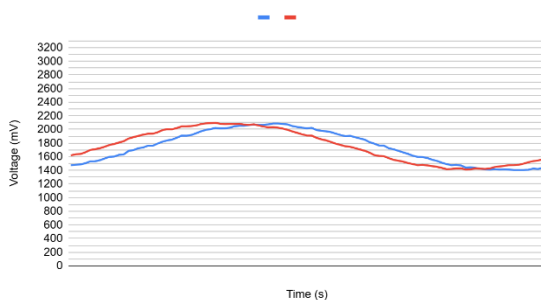


Figure 28 The output of my op-amp circuit for voltage, showing that the appropriate gain and offset is applied to the input signal.

5.4 Calibration test for ADC

Using the debugger, I was able to get the voltage buffer and current buffer values, which I formatted into columns in Google Sheets. I then created the following graphs, which showed that the ADC was sampling a proper sine wave.

500mVpp C&V ADC data with 30 degree phase



2Vpp C&V ADC data.

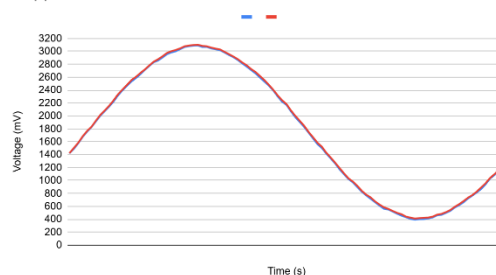


Figure 29 Calibration of voltage (Blue sinusoid) and current (Red sinusoid)

The blue graph depicts a voltage waveform and the red graph depicts a current waveform. You can see that the voltage waveform leads by 30 degrees which shows that the phase is also represented in the sampling of the waveform. The peak to peak is about 600mVpp read from the graph. This gives us a $gain = \frac{600}{500} = 1.20$ which corresponds with the measured gain in table 5.3.1.

5.5 Voltage, current, Phase, P, Q and S measurements

Here we use the UART “*Stat#\n” command output to calculate the difference between the theoretical measured voltage, current and apparent Power. As well as the error

```

*Stat#
2025/02/26 22:13:04
Voltage: 58.6V
Current: 3744mA
Phase: 0.000rad
Apparent: 219VA
Real: 219W
Reactive: 0VAR
PowerFact: 1.000
Energy/d: 0Wh
MaxPower: 273W
UnitsLeft: 10.0kWh

*Stat#
2025/02/26 22:33:33
Voltage: 115.9V
Current: 7538mA
Phase: 0.000rad
Apparent: 874VA
Real: 874W
Reactive: 0VAR
PowerFact: 1.000
Energy/d: 0Wh
MaxPower: 884W
UnitsLeft: 9.9kWh

*Stat#
2025/02/26 22:36:36
Voltage: 171.8V
Current: 11138mA
Phase: 0.000rad
Apparent: 1913VA
Real: 1913W
Reactive: 0VAR
PowerFact: 1.000
Energy/d: 0Wh
MaxPower: 1937W
UnitsLeft: 9.9kWh

*Stat#
2025/02/26 22:39:27
Voltage: 227.5V
Current: 14794mA
Phase: 0.000rad
Apparent: 3366VA
Real: 3366W
Reactive: 0VAR
PowerFact: 1.000
Energy/d: 0Wh
MaxPower: 3428W
UnitsLeft: 9.8kWh

*Stat#
2025/02/26 22:41:26
Voltage: 251.9V
Current: 16233mA
Phase: 0.000rad
Apparent: 4089VA
Real: 4089W
Reactive: 0VAR
PowerFact: 1.000
Energy/d: 0Wh
MaxPower: 4109W
UnitsLeft: 9.7kWh

```

Input	Vrms (theory) (V)	Vrms measured (V)	Error V	Irms(theory) (mA)	Irms measured (mA)	Error I	Apparent (theory)(SA)	Apparent measured (SA)	Error SA
0,5	57,5	58,7	2,04%	3750	3744	0,16%	215,625	219	1,565%
1	115	115,9	0,78%	7500	7538	0,51%	862,5	874	1,35%
1,5	172,5	171,6	0,52%	11250	11138	1%	1940,625	1913	1,4%
2,2	250	250,78	0,3%	16500	16394	0,6%	4125	4089	0,8%
2	230	227,5	1,08%	15000	14794	1,37%	3450	3366	2,434%

Table 2 Voltage, current and apparent power calculations.

The voltage,current, and apparent power measured results are within the 5% tolerance specified in the PDD. The error increases as the peak-to-peak input signal decreases, and this is expected as the signal resolution is not good with small input signals.

I used the “*Stat#\n” command with different phases for phase measurements as well as different input signal sizes. This shows the accuracy of my phase and that is it within 5% tolerance.

Phase input on ocilloscope.	Theoretical phase rad	0.5Vpp Phase (rad)	Error 0.5vpp %	1Vpp Phase (rad)	Error 1Vpp %2	2Vpp Phase(rad)	Error 2Vpp %3
-90	1,57079	1,5972	1,67	1,506	4,1	1,573	0,13
-30	0,523598	0,5362	2,41	0,5285	0,93	0,528	0,84
0	0	0	0	0	0	0	0
30	-0,523598	-0,50753	3,07	-0,515	1,64	-0,521	4,96
90	-1,57079	-1,555	1	-1,567	2,4	-1,5607	6,4

Table 3 Difference phase measurements(on the next page)

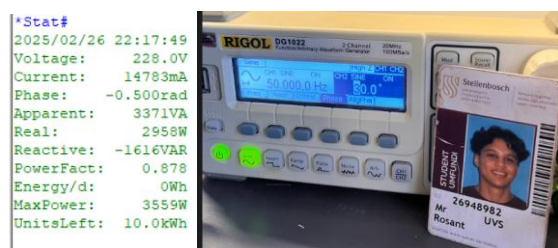


Figure 30 Phase measurements

Measurements for my Real Power, Reactive Power, and Power factor were as follows. I used a constant 30° phase on all my measurements and only changed the input signal size to see how accurate my measurements were to the theoretically calculated values.

- Real Power = Apparent × cos(30)
- Reactive power = Apparent × sin(30)

Input	Real Power Theory (W)	Reactive Power theory (VAR)	PF (rad) theory	Real Power measured (W)	Reactive Power measured (VAR)	PF measured (rad)
0,5	186,7	-107,81	0,866	189	-106	0,8717

1	746,95	-431,25	0,866	750	-418	0,873
1,5	1680	-970	0,866	1666	-945	0,869
2	2987,79	-1725	0,866	2958	-1616	0,878

Table 4 Accuracy of the Real & reactive power.

We can see that the measured real and reactive power is within 5% tolerance as specified. This shows that my system is consistent with a wide range of signals.

5.6 Load circuit measurements

Using a multimeter, I tested the gate voltage of the NMOS transistor as well as the voltage drop across the relay coils to see if the circuit was working as intended. The intended gate voltage that we required to turn on the transistor is any voltage between 0.8V-3V. the measured gate voltage is 3.09V, which is close to the calculated gate voltage of 3.21V. The voltage across the coils of the inductor when the NMOS circuit is turned on should be 5V because it is being supplied with a 5V power supply from the 5V rail. The measured voltage across the coils of the relay is 4,73V as there is a small resistance present from the diode as well as the relay circuit.

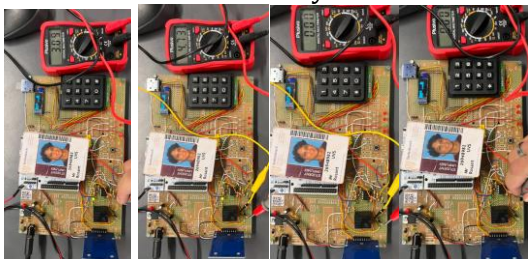


Figure 31 The gate voltage when on (1st figure), coil voltage drop when on (2nd figure), gate voltage when off (4th figure), coil voltage drop when off (3rd figure)

5.7 Keypad and button

To test that I was using the correct debouncing time, I tested the bouncing of the S3 button as well as the 4x3 matrix keypad buttons. This will ensure that I do not get accidental button presses. The procedure regarding the testing of the buttons was as follows:

- S3: Use an oscilloscope probe and connect it to the PA8 GPIO.
- Keypad: Set one PC0 high so that Row 1 on the 4x3 matrix keypad is active. Now connect the oscilloscope to the 4th Column pin from the right. This coincides with Column 1 on the keypad matrix.

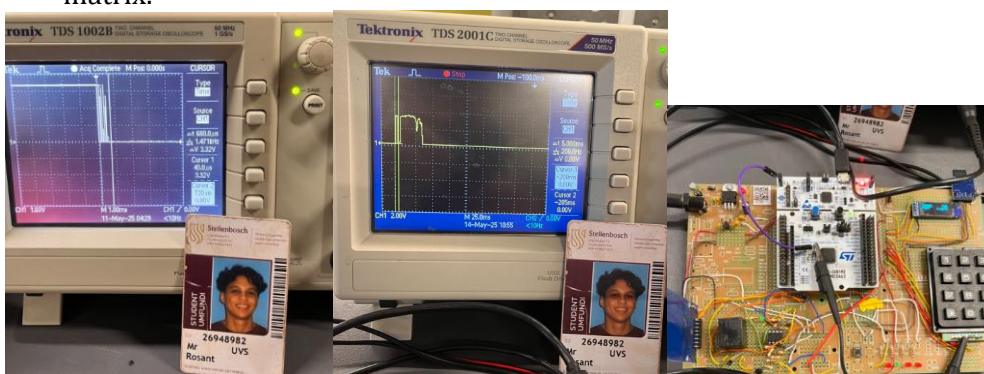


Figure 32 The S3 button bounce (1st figure), the keypad "1" key (2nd figure), Test setup for keypad (3rd figure)

The longest bouncing time I recorded was 5ms therefore, we are assured that a 50ms debouncing time for the keypad is sufficient and will not result in any extra keypresses. For the S3 button 10ms is sufficient as the longest bounce measured on the S3 button was 700 μ s. I can conclude that the button and keypad does not bounce much at all.

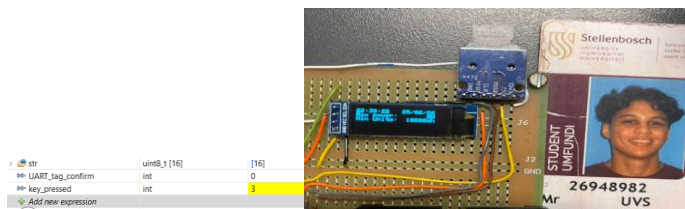


Figure 33 debugger test

Using the Debugger in the STM32CubeIDE I was able to check which key was pressed by going into the live expressions and typing the global keypressed variable into the expressions table. I then pressed “3” key, which corresponds to the menu on the right. This verified that the keypad was being read correctly.

5.8 OLED functionality

Test if the correct data is being displayed on the OLED on power-up. The method I used to test the OLED was as follows:

- Connect an oscilloscope probe to the SDA line, and when data is being transmitted to the OLED via I2C, the MCU will pull the SDA line low and transmit the Data needed to be written on the OLED display.
- Using a figure, I verified that the OLED was displaying the correct data of the default page on start-up.



Figure 34 SDA line pulled low (1st figure), Test setup (2nd figure), default display menu(3rd figure), specifications display page (4th figure).

5.9 RFID functionality

Using the tag, we were given to turn on counting we were able to extract the unique tag code, which is read to verify that the correct RFID tag was presented and turn on counting or add units. Using live expressions in the debugger, I was able to read the unique ID code into a variable called sNum in which I stored the Unique ID. I used termite to send a “*Units00210#\n” command which will add 0.210 Units to the system only if the RFID is present. After the RFID is presented it will update the Units to 10.220kWh.

- The following sequence of images depicts the process in which to add units as well as the RFID to be present by using UART instead of using the keypad, which takes more keypresses.

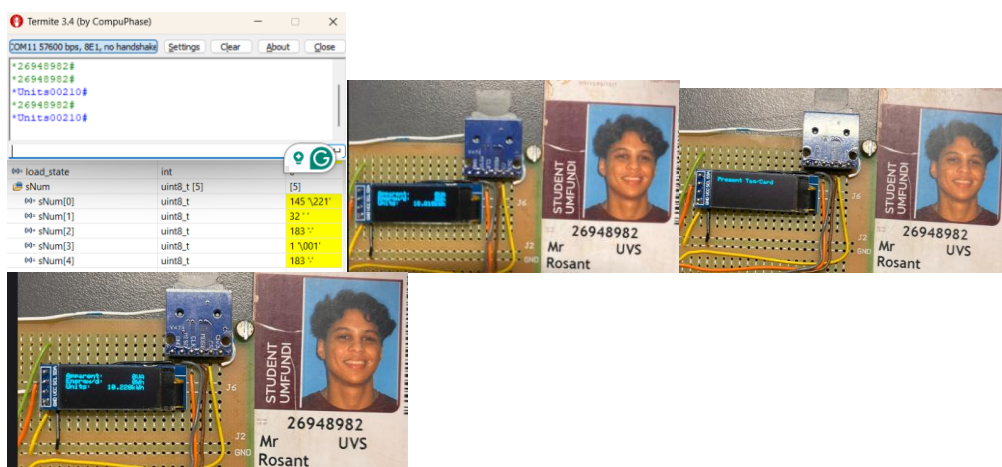


Figure 35 STM debugger and UART(1st figure),default page before units added(2nd figure),present tag page,(3rd figure),default page after units have been added(4th figure).

5.10 SD card

Testing if my SD card was displaying the correct data as well as logging to the csv file each second. This proves that my SD card was working as intended.

- I used termite and the UART “*Log#\n” and “*Dump#\n” to display the data which was logged on the SD-card.
- Using an oscilloscope probe, I connected to the CS pin on the SD card module. On the oscilloscope, we were able to see that the SD card was written to every second using the CUSOR function and measuring the difference between each time the CS line was pulled down.

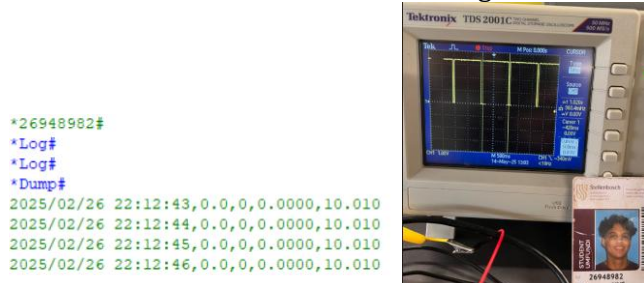


Figure 36 Termite display (Left), Logging each second (Right)

5.11 Menu functionality

Using the live expression viewer, I was able to track which STATE the OLED should be in due to a keypress and this aligns with the State diagram I created for my OLED menu functionality. (my “*” and “#” is mapped as the integer 10 and 11 respectively key for simplicity.

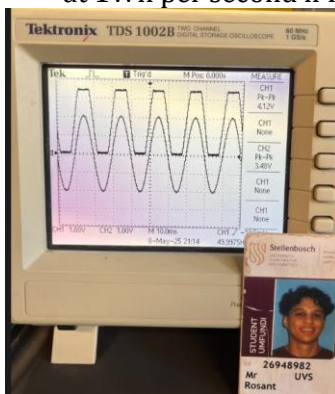
OLED	Key pressed	Key sequence
	key_pressed: int 1 current_state: DisplayState STATE_DISPLAY_1	1: display page 1.
	key_pressed: int 2 current_state: DisplayState STATE_DISPLAY_2	2: display page 2.
	key_pressed: int 3 current_state: DisplayState STATE_DISPLAY_3	3: display page 3.
	key_pressed: int 0 current_state: DisplayState STATE_DEFAULT	0: default page
	key_pressed: int 10 current_state: DisplayState STATE_MENU_LVL1	*: menu page
	key_pressed: int 1 current_state: DisplayState STATE_LOAD_MENU_FIRST	*>1: Load menu
	key_pressed: int 1 current_state: DisplayState STATE_LOAD_MENU key_pressed: int 2 current_state: DisplayState STATE_LOAD_MENU	*>1>1or2: Load select menu
	key_pressed: int 2 current_state: DisplayState STATE_MENU_LVL2	*>2: menu 2
	key_pressed: int 1 current_state: DisplayState STATE_UNIT_COUNT_MENU_FIRST	*>2>1: count menu
	key_pressed: int 1 current_state: DisplayState STATE_UNIT_COUNT_MENU key_pressed: int 2 current_state: DisplayState STATE_UNIT_COUNT_MENU	*>2>1>1or 2: count select menu
	key_pressed: int 2 current_state: DisplayState STATE_ADD_UNITS_MENU	*>2>2: Units add menu enter numbers 0-9
	key_pressed: int 10 current_state: DisplayState RFID_Tag_units	*>2>2>xxxxx>*: RFID authentication display

Table 5 Menu functionality

The above table shows the menu flow with the different key sequences. It also shows that the correct menu is displayed as well as the last key in the sequence is displayed. The '#' key however, there is no measurement showing it returning to a previous menu as you can use the '#' key in any one of the menus to display the previous state.

5.12 Full system Test

- Power supply: 5V and 3.3V output rails as expected.
- LEDs: All debug LEDs flashed as expected. Load simulating LED turns on as expected when s3 switch is pressed, as well as UART "*Load#\n" command being sent on termite.
- UART: works as intended using different commands such as "*Log#\n", "*Dump#\n", "*Stat#\n" etc. Displaying the correct data on UART or logging correctly to the SD card.
- Analog measurements: System provided the correct V,I, and power values for the simulating input signal. I tested this by checking how errors there was from the theoretical calculations.
- User Interface: The RFID and keypad work as intended. The OLED also displays the correct output on the display for certain key sequences. RFID authenticates added units if the correct tag is presented.
- SD card and Energy metering: Data logged each second as intended. Visually see units decrement at 1Wh per second if Real power is 3600W.



Software flowed as intended. There were no hardware failures or memory corruptions during the program's flow. Stress tested the Op-amp circuit to see if input signals greater than 3V would cause clipping and not breaking the ADC. Figure 5.12.1 shows how the signal is clipped if a large input signal is applied.

Figure 37 Clipping Op-amp signal

6 Conclusions

The overall system works as intended. The Power regulators correctly outputs 5V and 3.3V consistently. Debug LED's flash as expected for different Energy requirements e.g low units, high power usage etc. V/I sensing circuit samples waveforms accurately and timely although there are numerous other tasks the CPU must prioritize. The subsequent voltage, current and power calculations were in 5% tolerance. The system can receive commands both from the UART and settings chosen on the OLED. Display the correct information from those commands. The keypad and OLED interface was streamlined and consistent there were little to no double keypresses on the keypad and with the different key sequences input. The OLED displayed the correct page and worked as intended.

The Energy metering was accurate, and Units were kept track of using a SD-card. All commands related to the SD-card functioned as intended, however has some formatting issues with how text was stored on the SD-card. RFID interface was consistent and provided the needed security. Adding Units or toggling count on/off needed the RFID. Load circuit operated as intended and could be toggled on/off consistently without any delays.

6.1 Improvements

If we received all components at the start of the semester of an indication of the size of the components and its functionality a better layout on the baseboard could have been achieved which would make it more presentable.

The RFID could have been placed elsewhere as the EM effects from the antenna on board interfered with the V/I sensing circuit, which caused noise on the 3.3V rail. This caused some inaccuracies If smaller input signals were applied to the board. Adding more decoupling capacitors could filter out the noise from all components in general.

For V_{rms} and I_{rms} calculations are dependent on the peak-peak voltages and can be affected by transients. I would focus on calculating the V_{rms} and I_{rms} by using the formula for root-mean square and get the peak-to-peak voltages from that. As this is less affected by transients.

$$Eq\ RMS = \sqrt{\frac{1}{100} \sum_0^{100} (samples)^2}$$

This will allow my system to be more accurate and consistent with voltage and current measurements.

7 Appendix:

7.1 Appendix A:

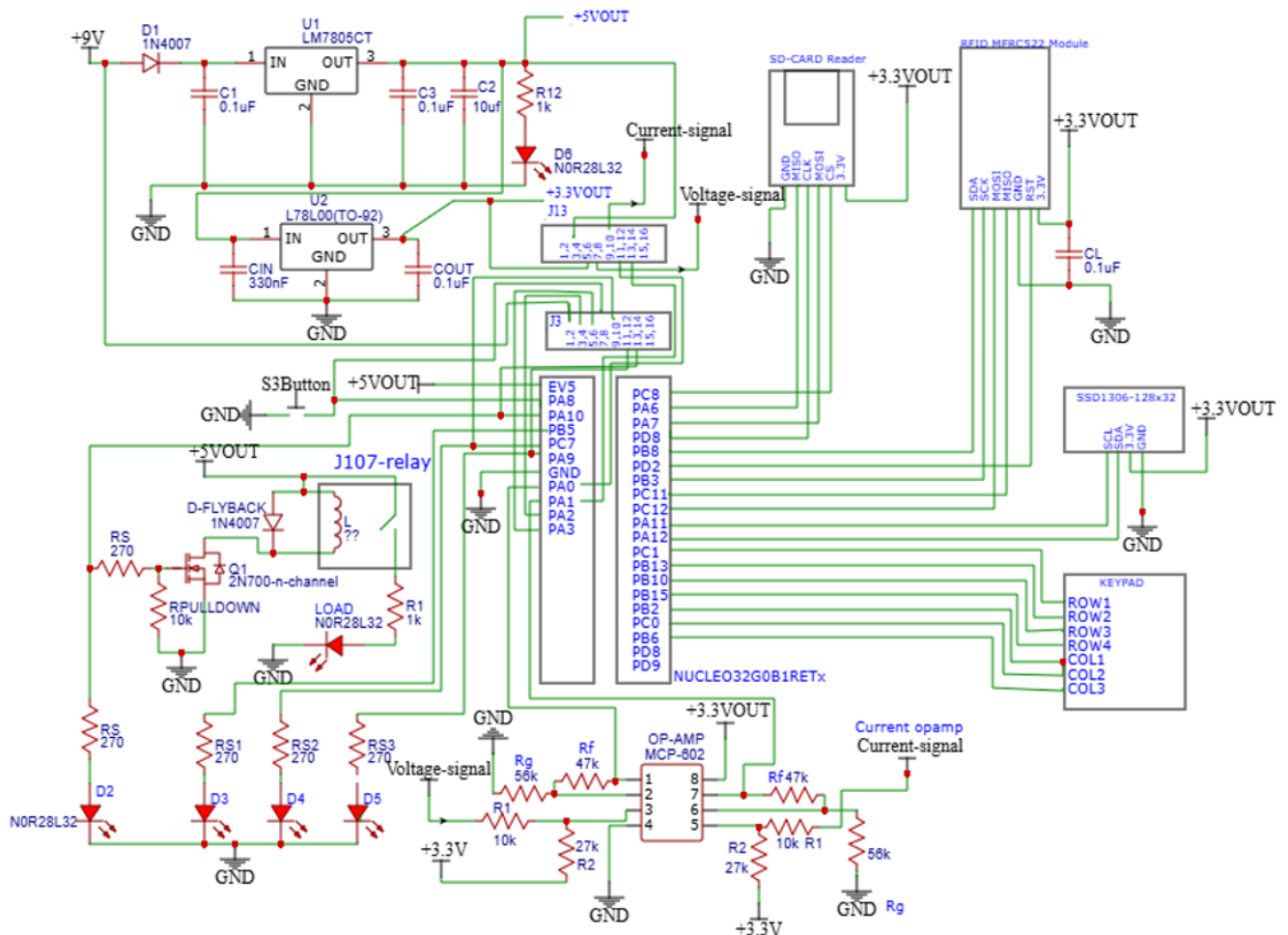


Figure 38 Full SPEC schematic

7.2 Pin out configurations

The following pins were used in my program and I have the reasoning for each pin selection [8]

PIN	Circuit Component	Reason for Pin Connection and configuration
PA0	Op-amp output voltage	Voltage output from the Op-amp circuit, directly fed into ADC1(channel 0), no pull-up or pull down resistor. Analog mode.
PA1	Op-amp output current	Current output from the Op-amp circuit, directly fed into ADC1(channel 1),no pull-up or pull down resistor. Analog mode.
PA2	USART2 (TX)	Configured as the transmit pin for USART2 to send data (e.g., student number, status reports) to the test station via serial communication.
PA3	USART2 (RX)	Configured as the receive pin for USART2 to receive commands (e.g., "**Stat#\n", "**Unitsxxxx#\n") from the test station.
PA5	Onboard green LED	Automatically configured when ioc created.
PA6	SD Card (MOSI)	Connected to the MOSI pin of the SD card module for SPI1 communication to send data to the SD card. Internal Pull-up resistor. Alternate Function Push-Pull.
PA7	SD Card (MISO)	Connected to the MISO pin of the SD card module for SPI1 communication to receive data from the SD card. Internal Pull-up resistor. Alternate Function Push-Pull.
PA8	Emergency S3 Button	Configured as an external interrupt input with falling edge detection for the active-low emergency button to toggle the load. Internal Pull-up resistor.
PA9	D5 LED	Configured as a GPIO output to drive D5 LED and flash for 500ms when power exceeds 3000W as a high power usage warning. Output Push-pull.
PA10	Relay Control / LED (D2)	Configured as a GPIO output to drive the 2N7000 MOSFET for relay control and to indicate load status via LED D2. Output Push Pull
PA11	OLED (SCL)	Connected to the SCL pin of the SSD1306 OLED display for I2C2 communication to carry the clock signal. Alternate Function Open Drain, Internal Pull-up resistor.
PA12	OLED (SDA)	Connected to the SDA pin of the SSD1306 OLED display for I2C2 communication to carry data. Alternate Function Open Drain, Internal Pull-up resistor.
PA13	SYS_SWIO	Automatically configured when ioc created.
PA14	SYS_SWCLK	Automatically configured when ioc created.
PB2	Keypad (Column)	Configured as a GPIO input with external interrupt, rising edge detection to detect key presses in the keypad's column 2. Internal Pull-down resistor.
PB3	RFID (CLK)	Connected to the CLK pin of the MFRC522 RFID module for SPI3 communication to synchronize data transfer. Internal Pull-up resistor. Alternate Function Push-Pull.
PB5	LED (D3)	Configured as a GPIO output to drive LED D3, which flashes every 500 ms to indicate low energy units. Output Push-Pull
PB6	Keypad (Column)	Configured as a GPIO input with external interrupt, rising edge trigger detection to detect key presses in the keypad's column 3. Internal Pull-up resistor.
PB8	RFID (SDA/CS)	Connected to the SDA (Chip Select) pin of the MFRC522 RFID module for SPI3 communication. Output Push-Pull. Internal Pull-Up resistor. GPIO Output.
PB10	Keypad (Row)	Configured as a GPIO output to drive row 2 of the 4x3 keypad matrix. Output push-pull configuration.
PB13	Keypad (Row)	Configured as a GPIO output to drive row 3 of the 4x3 keypad matrix. Output push-pull configuration.
PB15	Keypad (Row)	Configured as a GPIO output to drive row 4 of the 4x3 keypad matrix. Output push-pull configuration.
PC0	Keypad (Column)	Configured as a GPIO input with external interrupt, rising trigger detection to detect key presses in the keypad's column 1. Internal Pull-down resistor.

PC1	Keypad (Row)	Configured as a GPIO output to drive row 1 of the 4x3 keypad matrix. Output push-pull configuration.
PC7	D4 LED	Configured as a GPIO output to drive D4 LED and flash on each unit decrement. Output push pull.
PC8	SD Card (CS)	Connected to the Chip Select (CS) pin of the SD card module for SPI1 communication to enable/disable the SD card. Output Push-Pull. Internal Pull-up resistor.
PC11	RFID (MISO)	Connected to the MISO pin of the MFRC522 RFID module for SPI3 communication to receive data from the RFID module. Internal Pull Up resistor. Alternate Function Push-Pull.
PC13	Blue_push button	No pull-up or pull-down resistor, external interrupt with rising edge detection.
PC14	RCC_OSC32_IN	Automatically configured when ioc created.
PC15	RCC_OSC32_OUT	Automatically configured when ioc created.
PC12	RFID (MOSI)	Connected to the MOSI pin of the MFRC522 RFID module for SPI3 communication to send data to the RFID module. Internal Pull-up resistor. Alternate Function Push-Pull.
PD2	RFID (RST)	Connected to the Reset pin of the MFRC522 RFID module to control the module's reset state. Output Push-Pull. Internal Pull-up resistor.
PD8	SD Card (CLK)	Connected to the CLK pin of the SD card module for SPI1 communication to synchronize data transfer. Internal Pull-up resistor. Alternate Function Push-Pull.
EV5	From 5V regulator	Connected to the 5V regulator to give your board power externally and not from STlink
GND	Ground	Connected to the baseboard global ground.
PF0	RCC_OSC_IN	Automatically configured when ioc created.

Table 6 PIN out Table.

8 References

- [1] Alebakwe, "How to use RFID RC522 with STM32 | EASY TUTORIAL | STM32CUBEIDE," YouTube, 22-Jan-2025.[Online].Available: <https://www.youtube.com/watch?si=CXF0WODxikcvwtpR&v=PBIm8BCnbyQ&feature=youtu.be>.
- [2] kiwih, "cubeide-sd-card," GitHub, 2025. [Online]. Available: <https://github.com/kiwih/cubeide-sd-card/tree/master>.
- [3] CircuitGatorHQ, "RFID-RC522-with-STM32," GitHub, 2025. [Online]. Available: <https://github.com/CircuitGatorHQ/RFID-RC522-with-STM32>.
- [4] Alebakwe, "STM32CubeIDE Programming, Debugging, and Running," YouTube, 26-Jul-2021. [Online]. Available: <https://youtu.be/Ay3ysGDrxbo?si=m8Qz1bMOL1zunD-u>.
- [5] Microchip Technology Inc., "MCP601/1R/2/3/4 2.7V to 6.0V Single Supply CMOS Op Amps," Datasheet DS21314G, Chandler, AZ, USA, Dec. 2007. [Online]. Available: <http://www.microchip.com>
- [6] B. Carter, "Designing Gain and Offset in Thirty Seconds," Texas Instruments, Dallas, TX, USA, Appl. Rep. SLOA097, Feb. 2002. [Online]. Available: <http://www.ti.com>

- [7] Dr A. Barnard and Dr A. du Plessis, "Design (E) 314 PDD: Smart Prepaid Energy Controller," Stellenbosch Univ., Stellenbosch, South Africa. v0.9d, May 2025.
- [8] STMicroelectronics, "STM32 Nucleo-64 boards (MB1360) user manual," STMicroelectronics, Geneva, Switzerland, Tech. Rep. UM2324 Rev. 5, Jul. 2024.
- [9] STMicroelectronics, "STM32G0B1xB/xC/xE: Arm Cortex-M0+ 32-bit MCU, up to 512KB Flash, 144KB RAM," STMicroelectronics, Geneva, Switzerland, Datasheet DS13560 Rev. 5, Jun. 2024. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32g0b1re.pdf>.
- [10] A. Fiskin, "stm32-ssd1306: Library for SSD1306 OLED display," GitHub, 2020. [Online]. Available: <https://github.com/afiskon/stm32-ssd1306>. [Accessed: May 18, 2025].
- [11] P. Tech, "STM32 I2C tutorial with SSD1306 OLED," YouTube, Jun. 12, 2022. [Online]. Available: <https://www.youtube.com/watch?v=z1Px6emHleg&t=774s>
- [12] SparkFun Electronics, "12-Button Keypad COM-14662," SparkFun Electronics, Boulder, CO, USA, Datasheet, n.d. [Online]. Available: <https://www.sparkfun.com/products/14662>.
- [13] STMicroelectronics, "L7800 series: Positive voltage regulators," STMicroelectronics, Geneva, Switzerland, Datasheet, Aug. 2006. [Online]. Available: <https://www.st.com>.
- [14] STMicroelectronics, "L78LxxAB, L78LxxAC, L78LxxC: Positive voltage regulators," STMicroelectronics, Geneva, Switzerland, Datasheet, Apr. 2009. [Online]. Available: <https://www.st.com>.
- [15] CIT Relay & Switch, "J107 Series: General purpose relay," CIT Relay & Switch, Minneapolis, MN, USA, Datasheet, Jan. 2023. [Online]. Available: <https://www.citrelay.com>.
- [16] Britek (Europe) Limited, "NOR28L32: 3.0mm Round Type LED Lamp," Britek (Europe) Limited, Datasheet, 2023. [Online]. Available: <https://www.brightekeurope.com>.
- [17] HandsOn Technology, "RC522 RFID Development Kit," HandsOn Technology, Datasheet MDU1040, 2023. [Online]. Available: <https://www.handsontec.com>.
- [18] Microchip Technology Inc., "2N7000: N-Channel Enhancement-Mode Vertical DMOS FET," Microchip Technology Inc., Chandler, AZ, USA, Datasheet DS20005695A, Feb. 2021. [Online]. Available: <https://www.microchip.com>.